

Setting Up VS Code, Git, and LaTeX on Windows: A Complete Beginner's Guide

Contents

1 Quick Checklist	1
2 Git for Windows	2
3 GitHub Account	3
4 GitHub Desktop	4
5 Visual Studio Code (VS Code)	5
6 LaTeX Distribution: MiKTeX	6
7 Strawberry Perl (Optional)	7
8 Recommended VS Code Extensions	7
9 First-Use Workflows	9
10 Troubleshooting	11
11 Where to Go Next	12

By the end of this guide you will have a fully working environment on your Windows computer where you can:

- Write and compile LaTeX documents (papers, dissertations, slides) without leaving your editor.
- Track every change to your files with Git and back them up to GitHub.
- Edit and run R scripts alongside your LaTeX projects.
- Collaborate with co-authors through GitHub, with a visual interface that requires no command-line expertise.

You do not need to abandon RStudio. Think of VS Code as a companion: it excels at LaTeX editing, Git integration, and working across multiple languages in one window. For heavy R work — interactive data exploration, plotting, package development — RStudio remains an excellent choice. Many researchers keep both open and use whichever fits the task at hand.

This guide assumes you are running Windows 10 or Windows 11 (64-bit) and that you already have R and RStudio installed. Everything else, we will set up together, step by step.

1 Quick Checklist

Here is everything you will install, in order. Budget roughly 60 to 90 minutes for the full setup if your internet connection is reasonable.

1. **Git for Windows** — the version-control engine (~5 minutes)
2. **GitHub account** — your free cloud home for repositories (~5 minutes)
3. **GitHub Desktop** — a visual interface for Git (~5 minutes)
4. **Visual Studio Code (VS Code)** — your new code and LaTeX editor (~5 minutes)
5. **MiKTeX** — the LaTeX distribution that compiles your .tex files (~15 minutes, depending on download speed)
6. **Strawberry Perl** (*optional*) — only needed if you want to use the latexmk build tool (~5 minutes)
7. **VS Code extensions** — add-ons that teach VS Code about R, LaTeX, and Git (~10 minutes)

After the installations, three short tutorials will walk you through your first real tasks.

2 Git for Windows

2.1 Purpose

Git is a version-control system. It keeps a complete history of every change you make to your files, lets you undo mistakes, and makes it possible to collaborate with others without overwriting each other's work. Every tool in this guide builds on Git.

2.2 Download

Go to the official Git website and download the installer:

<https://git-scm.com/download/win>

The site should detect your system and offer the 64-bit installer automatically. Click the download link for the “Standalone Installer” (64-bit).

2.3 Installation

Run the downloaded .exe file. If Windows asks “Do you want to allow this app to make changes to your device?”, click **Yes** — this is a standard administrator permission prompt that most installers require.

The installer has many screens. Here are the recommended choices:

1. **Select Components** — leave the defaults checked. Make sure “Windows Explorer integration” is checked so you can right-click folders later.
2. **Choosing the default editor used by Git** — select **Notepad** (or VS Code if you want, but we have not installed it yet). The default option, Vim, is powerful but confusing for beginners.
3. **Adjusting the name of the initial branch** — select “**Override the default branch name**” and type main. This matches GitHub's current convention.
4. **Adjusting your PATH environment** — select the recommended option: “**Git from the command line and also from 3rd-party software**”. This adds Git to your system PATH, which means other programs (VS Code, GitHub Desktop) can find it.
5. **Choosing the SSH executable** — leave the default (Use bundled OpenSSH).
6. **Choosing HTTPS transport backend** — select “**Use the native Windows Secure Channel library**”. This lets Git use Windows' built-in credential storage.
7. **Configuring the line ending conversions** — select “**Checkout Windows-style, commit Unix-style line endings**”. This is the safest option for collaborating across operating systems.
8. **Configuring the terminal emulator** — select “**Use Windows' default console window**”.
9. **Default behavior of git pull** — leave the default (fast-forward or merge).
10. **Choose a credential helper** — select “**Git Credential Manager**”. This saves your GitHub password so you do not have to type it repeatedly.
11. **Configuring extra options** — leave the defaults.
12. Click **Install** and wait for it to finish.

Note: If your antivirus software (Windows Defender or a third-party product) flags the installer or slows it down, this is a false positive. Git for Windows is safe. You may need to click “Allow” or temporarily pause real-time scanning.

2.4 A Brief Introduction to PowerShell

To verify that Git installed correctly, we need to open a terminal. Windows comes with a program called **PowerShell**, which lets you type commands that the computer executes. Think of it as a text-based way to tell your computer what to do.

To open PowerShell:

1. Press the **Windows key** on your keyboard (the key with the Windows logo, usually between Ctrl and Alt).
2. Type `powershell`.
3. Click **Windows PowerShell** in the search results.

A dark blue window will appear with a blinking cursor. This is where you type commands. After typing a command, press **Enter** to run it.

Note: Throughout this guide, when we say “open a terminal” or “run a command,” we mean: open PowerShell, type the command, and press Enter.

2.5 Verification

Open PowerShell and type:

```
git --version
```

You should see output similar to:

```
git version 2.43.0.windows.1
```

The exact version number does not matter as long as you see a version rather than an error message. If you see something like “‘git’ is not recognized as an internal or external command,” Git did not install correctly or was not added to your PATH — try restarting your computer and running the command again.

2.6 Initial Configuration

While you have PowerShell open, tell Git your name and email. These are attached to every change you make so collaborators know who did what. Run these two commands, replacing the placeholder text with your own information:

```
git config --global user.name "Your Full Name"
git config --global user.email "your.email@example.com"
```

Use the same email address you will use for your GitHub account in the next step.

3 GitHub Account

3.1 Purpose

GitHub is a website that hosts Git repositories (project folders) in the cloud. It serves as your backup, your collaboration platform, and your public portfolio of research code and documents.

3.2 Creating Your Account

1. Go to <https://github.com> in your web browser.
2. Click **Sign up**.

3. Follow the prompts: enter your email, create a password, and choose a username. A professional username (such as your name or initials) is ideal since collaborators and reviewers may see it.
4. Complete the verification puzzle and confirm your email address.

The free tier is all you need. It includes unlimited public and private repositories, which covers every use case in this guide.

Note: Remember the email address you used — it should match the one you set in `git config --global user.email` in the previous section.

4 GitHub Desktop

4.1 Purpose

GitHub Desktop is a graphical application that lets you use Git without typing commands. You can clone repositories, make commits, push changes, and manage branches through a visual interface. It is especially helpful when you are learning Git, because you can see exactly what is happening at each step.

4.2 Download

Go to the official GitHub Desktop website:

<https://desktop.github.com>

Click the **Download for Windows (64bit)** button.

4.3 Installation

1. Run the downloaded installer. GitHub Desktop installs into your user folder and does not require administrator permissions.
2. When the application opens, click **Sign in to GitHub.com**.
3. Your web browser will open. Authorize GitHub Desktop to access your account.
4. Back in GitHub Desktop, confirm your Git configuration (name and email). These should already be filled in from the `git config` commands you ran earlier.

4.4 Configuration

After signing in:

1. Go to **File > Options** (or press `Ctrl + ,`).
2. Under the **Integrations** tab, set **External Editor** to **Visual Studio Code**. (If VS Code is not installed yet, come back to this step after installing it in the next section.)
3. Under the same tab, confirm that **Shell** is set to **PowerShell**.

4.5 Verification

You should now see the GitHub Desktop dashboard. If it shows “Let’s get started!” with options to create or clone a repository, everything is working.

5 Visual Studio Code (VS Code)

5.1 Purpose

Visual Studio Code is a free, open-source code editor made by Microsoft. It is lightweight but powerful, with extensions that add support for virtually any programming language or document format. With the right extensions, it becomes an excellent LaTeX editor and a capable R environment, while also giving you the best Git integration of any editor.

VS Code is complementary to RStudio. Use RStudio when you want its specialized R features (interactive plots, environment browser, package manager). Use VS Code when you are writing LaTeX, working across multiple languages, or want deeper Git integration. Many researchers keep both installed and switch between them depending on the task.

5.2 Download

Go to the official VS Code website:

<https://code.visualstudio.com>

Click the large **Download for Windows** button. This gives you the User Installer (64-bit), which is the right choice for most people.

5.3 Installation

Run the downloaded installer and follow these steps:

1. Accept the license agreement.
2. On the **Select Additional Tasks** screen, check all of the following:
 - **Add “Open with Code” action to Windows Explorer file context menu** — this lets you right-click any file and open it in VS Code.
 - **Add “Open with Code” action to Windows Explorer directory context menu** — same for folders.
 - **Register Code as an editor for supported file types.**
 - **Add to PATH (requires shell restart)** — this is important. It lets you type code in PowerShell to open VS Code.
3. Click **Install** and wait.

Note: The “Add to PATH” option means that after installation, the word `code` becomes a command your computer recognizes. But this only takes effect after you close and reopen any PowerShell windows. If `code --version` does not work immediately, close PowerShell and open it again.

5.4 Verification

Close any open PowerShell windows, then open a new one and type:

```
code --version
```

You should see something like:

```
1.87.0
e54c774e0add60467559eb0d1e229c3e4b9cf0f7
x64
```

The version number and hash will differ, but as long as you do not see an error, VS Code is installed and on your PATH.

5.5 A Quick Tour of the Interface

When you open VS Code for the first time, here is what you will see:

- **Activity Bar** (left edge) — icons for Explorer (files), Search, Source Control (Git), Extensions, and more.
- **Side Bar** — the panel that opens when you click an Activity Bar icon.
- **Editor Area** (center) — where your files appear in tabs.
- **Terminal Panel** (bottom, toggle with `Ctrl + ``) — a built-in PowerShell terminal, so you do not need to switch windows.
- **Status Bar** (very bottom) — shows your current Git branch, errors, and other status information.

To open Settings, press `Ctrl + ,` (Control and comma). You can search for any setting by name.

6 LaTeX Distribution: MiKTeX

6.1 Purpose

A LaTeX distribution is the software that turns your `.tex` source files into polished PDF documents. It includes the LaTeX compiler (`pdflatex`, `xelatex`, `lualatex`) and thousands of packages for formatting, bibliography management, diagrams, and more.

We recommend **MiKTeX** for Windows users. Compared to the other major distribution (TeX Live), MiKTeX has two advantages for beginners:

- **Smaller initial download.** MiKTeX installs a minimal set of packages and then downloads additional ones automatically the first time you use them. You do not need to download several gigabytes upfront.
- **Graphical package manager.** If you ever need to install or update a package manually, MiKTeX Console provides a point-and-click interface.

Note: TeX Live is a perfectly valid alternative, especially if you prefer a “download everything at once” approach. You can find it at <https://www.tug.org/texlive/>. The rest of this guide assumes MiKTeX, but the LaTeX Workshop extension in VS Code works with either distribution.

6.2 Download

Go to the official MiKTeX website:

<https://miktex.org/download>

Click the **Download** button for the Windows installer. Choose the “Basic MiKTeX Installer” (64-bit).

6.3 Installation

Run the downloaded installer:

1. If prompted for administrator permissions, click **Yes**.
2. **Installation scope** — select “**Install MiKTeX only for me**”. Installing for just your user account avoids permission issues later.
3. **Installation directory** — leave the default unless you have a reason to change it.
4. **Preferred paper size** — select **Letter** (US) or **A4** (international), depending on your usual document format.
5. **Install missing packages on-the-fly** — this is the key setting. Select “**Yes**”. This means that when you compile a document that needs a package you do not have, MiKTeX will download and install it automatically. This saves you from hunting down packages manually.
6. Click **Start** and wait for the installation to complete. This may take 10 to 15 minutes.

Note: After installation, MiKTeX may prompt you to run an update. Go ahead and do this — open **MiKTeX Console** from the Start menu, go to the **Updates** tab, and click **Check for updates**, then **Update now** if updates are available.

6.4 Verification

Open a new PowerShell window (close and reopen if one was already open) and type:

```
pdflatex --version
```

You should see output that starts with something like:

```
MiKTeX-pdfTeX 4.10 (MiKTeX 23.10)
```

If you see “‘pdflatex’ is not recognized,” MiKTeX may not be on your PATH yet. Try the following:

1. Open **MiKTeX Console** from the Start menu.
2. Go to **Settings** and look for a PATH-related option or click **Refresh PATH**.
3. Close and reopen PowerShell, then try again.

If it still does not work, restart your computer and try once more. Occasionally Windows needs a restart to recognize new PATH entries.

7 Strawberry Perl (Optional)

7.1 Purpose

Strawberry Perl is a Windows distribution of the Perl programming language. You only need it if you plan to use **latexmk**, a popular build tool that automates multi-pass LaTeX compilation (compiling your document, running BibTeX, then compiling again, and so on). The LaTeX Workshop extension in VS Code can use **latexmk** as its default build recipe.

If you are not sure whether you need this, you can skip it for now and come back later. The LaTeX Workshop extension also supports a simpler recipe that uses **pdflatex** directly, without Perl.

7.2 Download

<https://strawberryperl.com>

Click the download link for the 64-bit MSI installer.

7.3 Installation

1. Run the **.msi** installer.
2. Accept the defaults. Strawberry Perl will add itself to your PATH automatically.

7.4 Verification

Open a new PowerShell window and type:

```
perl --version
```

You should see output mentioning “Strawberry Perl” and a version number.

8 Recommended VS Code Extensions

Extensions are add-ons that give VS Code new capabilities. Below are the extensions that will turn VS Code into a powerful environment for R, LaTeX, and Git. To install an extension:

1. Open VS Code.
2. Click the **Extensions** icon in the Activity Bar (it looks like four small squares, on the left edge), or press `Ctrl + Shift + X`.
3. Type the extension name in the search box.
4. Click **Install** on the correct result.

8.1 R (REditorSupport.r)

Marketplace link: <https://marketplace.visualstudio.com/items?itemName=REditorSupport.r>

What it does: Adds R language support to VS Code — syntax highlighting, code completion, an integrated R terminal, and the ability to run R code line by line (similar to RStudio).

Configuration:

After installing, you need to tell VS Code where R is on your computer:

1. Press `Ctrl + ,` to open Settings.
2. Search for `r.rterm.windows`.
3. Set the value to the path of your R executable. This is usually something like:

`C:\Program Files\R\R-4.3.2\bin\x64\R.exe`

Adjust the version number to match what you have installed. You can find the correct path by opening RStudio, going to **Tools > Global Options > General**, and looking at the “R version” path.

Note: If VS Code cannot find R, the most common fix is correcting this path. See the Troubleshooting section at the end of this guide.

8.2 LaTeX Workshop (James-Yu.latex-workshop)

Marketplace link: <https://marketplace.visualstudio.com/items?itemName=James-Yu.latex-workshop>

What it does: This is the crown jewel for LaTeX in VS Code. It provides syntax highlighting, real-time error checking, one-click PDF compilation, a built-in PDF viewer, forward/inverse search (click in your LaTeX source and jump to that spot in the PDF, and vice versa), and autocomplete for LaTeX commands and bibliography entries.

Configuration:

LaTeX Workshop works out of the box with MiKTeX for basic documents. However, you may want to adjust the build recipe:

1. Press `Ctrl + ,` to open Settings.
2. Search for `latex-workshop.latex.recipe.default`.
3. If you installed Strawberry Perl and `latexmk` is available, the default recipe (`latexmk`) will work automatically.
4. If you did **not** install Perl, change the default recipe to use `pdflatex` directly:
 - Search for `latex-workshop.latex.recipes` in Settings.
 - You can add or reorder recipes. A simple recipe using only `pdflatex` looks like this in your `settings.json`:

```
"latex-workshop.latex.recipes": [  
  {  
    "name": "pdflatex",  
    "tools": ["pdflatex"]  
  }  
]
```

Note: You can also set LaTeX Workshop to build your document every time you save the file. Search for `latex-workshop.latex.autoBuild.run` in Settings and set it to `onSave`.

8.3 GitLens (eamodio.gitlens)

Marketplace link: <https://marketplace.visualstudio.com/items?itemName=eamodio.gitlens>

What it does: Supercharges the built-in Git features of VS Code. It shows who changed each line of code (and when), provides a rich history viewer, lets you compare versions of a file side by side, and makes branching and merging more visual.

Configuration: GitLens works immediately after installation. No additional setup is required.

8.4 GitHub Pull Requests and Issues (GitHub.vscode-pull-request-github)

Marketplace link: <https://marketplace.visualstudio.com/items?itemName=GitHub.vscode-pull-request-github>

What it does: Lets you manage GitHub pull requests and issues directly inside VS Code. You can review code, leave comments, and merge pull requests without switching to your browser.

Configuration:

After installing, VS Code will show a notification asking you to sign in to GitHub. Click **Sign in** and follow the prompts in your browser. This connects VS Code to the same GitHub account you set up earlier.

8.5 Markdown All in One (yzhang.markdown-all-in-one)

Marketplace link: <https://marketplace.visualstudio.com/items?itemName=yzhang.markdown-all-in-one>

What it does: Improves the experience of writing Markdown files (like README files, notes, and documentation). It adds keyboard shortcuts for formatting, a table of contents generator, live preview, and automatic list continuation.

Configuration: No additional setup is required.

8.6 Code Spell Checker (streetsidesoftware.code-spell-checker)

Marketplace link: <https://marketplace.visualstudio.com/items?itemName=streetsidesoftware.code-spell-checker>

What it does: Catches spelling mistakes in your code, comments, LaTeX documents, and Markdown files. It underlines misspelled words with a blue squiggle and suggests corrections. You can add technical terms and jargon to a custom dictionary so they stop being flagged.

Configuration: No additional setup is required. To add a word to your dictionary, right-click the underlined word and select “Add Word to User Dictionary.”

8.7 Path Intellisense (christian-kohler.path-intellisense)

Marketplace link: <https://marketplace.visualstudio.com/items?itemName=christian-kohler.path-intellisense>

What it does: Autocompletes file paths as you type them. When you start typing a file path in your code or LaTeX source (for example, `\includegraphics{figures/...}`), it shows a dropdown of matching files and folders. This eliminates typos in file references.

Configuration: No additional setup is required.

9 First-Use Workflows

Now that everything is installed, let’s put it to use. These three short tutorials will help you verify that your setup works and give you confidence with the basic workflows.

9.1 Workflow 1: Clone a GitHub Repository and Open It in VS Code

“Cloning” means downloading a copy of a repository from GitHub to your computer, with its full history intact.

1. Open **GitHub Desktop**.
2. Click **File > Clone Repository** (or press `Ctrl + Shift + O`).
3. Click the **URL** tab.
4. Paste the following URL into the “Repository URL” field — this is a small public test repository:
`https://github.com/github/hello-world`
5. Under “Local Path,” choose a folder on your computer where you want to save it (for example, `C:\Users\YourName\Documents\`).
6. Click **Clone**. GitHub Desktop will download the repository.
7. Once the clone finishes, click **Open in Visual Studio Code** (or go to **Repository > Open in Visual Studio Code**).
8. VS Code opens with the repository’s files in the Explorer side panel. You can click on any file to view and edit it.

You have just cloned your first repository. Any changes you make locally can be committed and pushed back to GitHub using GitHub Desktop.

9.2 Workflow 2: Compile a LaTeX Document in VS Code

1. Open VS Code (you can use `Ctrl + N` to create a new file).
2. Click **File > Save As** and save the file with the name `test.tex` in any folder you like.
3. Paste the following minimal LaTeX document into the file:

```
\documentclass{article}
\begin{document}
Hello, \LaTeX! This is my first document compiled from VS Code.
\end{document}
```

4. Save the file (`Ctrl + S`).
5. LaTeX Workshop should automatically compile the document. You will see activity in the bottom status bar. If it does not compile automatically, press `Ctrl + Alt + B` to trigger a build.
6. Once compilation finishes, press `Ctrl + Alt + V` to open the PDF preview panel inside VS Code. You should see a PDF with the text “Hello, LaTeX! This is my first document compiled from VS Code.”
7. Try editing the text (for example, change “first” to “second”), save the file, and watch the PDF update.

If compilation fails, check the “Problems” panel at the bottom of VS Code (press `Ctrl + Shift + M`). The most common issue is a missing LaTeX package, which MiKTeX should offer to install automatically. See the Troubleshooting section below if you run into difficulties.

9.3 Workflow 3: Run an R Script in VS Code

1. Open VS Code.
2. Create a new file and save it as `test.R`.
3. Type the following R code:

```
message("Hello from R in VS Code!")
x <- 1:10
mean(x)
```

4. Place your cursor on the first line.

5. Press `Ctrl + Enter` to send that line to the R terminal. An R terminal should open in the bottom panel, and you will see the output: `Hello from R in VS Code!`
6. Move to the next line and press `Ctrl + Enter` again. Repeat for the third line. You should see `[1] 5.5` as the result.
7. To select and run all lines at once, press `Ctrl + A` (select all) and then `Ctrl + Enter`.

If R does not start, VS Code may not know where to find it. See the Troubleshooting section for instructions on setting the R path.

10 Troubleshooting

10.1 LaTeX Package Not Found

Symptom: Compilation fails with an error like “File ‘somepackage.sty’ not found.”

Solution:

1. If MiKTeX’s on-the-fly installation is enabled (as recommended during setup), it should prompt you to install the missing package. Click **Install** when the dialog appears.
2. If no dialog appears, open **MiKTeX Console** from the Start menu.
3. Go to the **Packages** tab.
4. Search for the package name (without the `.sty` extension).
5. Right-click the package and select **Install**.
6. Return to VS Code and rebuild your document (`Ctrl + Alt + B`).

If the package still cannot be found, try running the MiKTeX update first (Updates tab in MiKTeX Console), then search again.

10.2 Git Authentication Failure

Symptom: When you try to push or pull, you see an error like “remote: Repository not found” or “fatal: Authentication failed.”

Solution:

1. Make sure you are signed in to GitHub Desktop. Open GitHub Desktop and check **File > Options > Accounts**.
2. In PowerShell, try running:

```
gh auth status
```

If it says you are not logged in, run:

```
gh auth login
```

Follow the prompts to authenticate through your browser.

3. If you are using Git from the command line and it keeps asking for your password, make sure Git Credential Manager is installed (it should be if you followed the Git installation steps above). You can verify by running:

```
git config --global credential.helper
```

It should say `manager` or `manager-core`.

10.3 R Interpreter Not Detected in VS Code

Symptom: When you try to run R code, VS Code says it cannot find an R interpreter, or the R terminal does not open.

Solution:

1. Press `Ctrl + ,` to open Settings in VS Code.
2. Search for `r.term.windows`.
3. Set the path to your R executable. Common locations include:
`C:\Program Files\R\R-4.3.2\bin\x64\R.exe`
`C:\Program Files\R\R-4.4.0\bin\x64\R.exe`
4. To find the exact path, open **File Explorer**, navigate to `C:\Program Files\R\`, and look for your version folder. Inside it, go to `bin\x64\` and you should see `R.exe`.
5. Copy the full path and paste it into the VS Code setting.
6. Restart VS Code after changing this setting.

10.4 Windows Defender or Antivirus Blocking an Installation

Symptom: An installer stalls, a downloaded file is quarantined, or you see a “Windows protected your PC” SmartScreen warning.

Solution:

1. **SmartScreen warning:** If you see a blue window that says “Windows protected your PC,” click **More info**, then click **Run anyway**. This appears because the installer is not commonly downloaded, not because it is dangerous. All software in this guide comes from official sources.
2. **Antivirus quarantine:** If your antivirus moves a downloaded file to quarantine, open your antivirus software, find the quarantined file, and restore it. Then temporarily disable real-time scanning while you run the installer.
3. **Windows Defender real-time protection:** If installation is extremely slow, you can temporarily pause real-time protection:
 - Open **Windows Security** from the Start menu.
 - Go to **Virus & threat protection > Manage settings**.
 - Toggle off **Real-time protection**. It will turn back on automatically after a while.
 - Run the installer, then toggle it back on manually if it has not re-enabled itself.

Note: Only disable real-time protection temporarily and only while installing software from the official sources listed in this guide. Always re-enable it afterward.

11 Where to Go Next

You now have a working environment for R, LaTeX, and Git. Here are resources to deepen your skills:

- **Happy Git and GitHub for the useR** by Jenny Bryan — an excellent, free online book aimed at R users who are new to Git. Covers the “why” as well as the “how.” <https://happygitwithr.com>
- **LaTeX Workshop documentation** — the complete reference for the VS Code extension, including advanced features like custom build recipes, multi-file projects, and snippet customization. <https://github.com/James-Yu/LaTeX-Workshop/wiki>
- **Visual Studio Code documentation** — the official guide to VS Code, including tips, keyboard shortcuts, and in-depth explanations of every feature. <https://code.visualstudio.com/docs>
- **Overleaf LaTeX tutorials** — if you are new to LaTeX itself (not just the tools), Overleaf has a well-written series of beginner tutorials. <https://www.overleaf.com/learn>

- **MiKTeX documentation** — for package management, configuration, and troubleshooting specific to MiKTeX.
<https://docs.miktex.org>

Take your time. You do not need to master everything at once. Start with whatever task is most pressing — compiling a paper, tracking changes to a project, or collaborating with a co-author — and learn the rest as you go.